

LAPORAN PRAKTIKUM
TUGAS 2 Konsep Container dan Instalasi Docker

Dosen Pengampu :
Ferry Astika Saputra



Disusun Oleh :
Adam Rasyid Nurmuhammad
3126640025
D4-IT-B

PROGRAM STUDI SARJANA TERAPAN
JURUSAN TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	3
BAB II LANDASAN TEORI	4
2.1 Containerization	4
2.2 Namespace.....	4
2.3 Cgroup	4
2.4 Docker	4
BAB III Pengerjaan Praktikum	5
3.1 Container Nginx dan Ubuntu interaktif.....	5
3.2 Dockerfile custom web statis	6
3.3 Analisis wajib:.....	7
BAB IV KESIMPULAN.....	9

BAB I PENDAHULUAN

1.1 Latar Belakang

Containerization merupakan teknologi yang memungkinkan aplikasi beserta seluruh dependensinya dikemas dan dijalankan dalam lingkungan yang terisolasi. Salah satu platform yang digunakan untuk menerapkan teknologi tersebut adalah Docker, yang menyediakan mekanisme untuk membuat, menjalankan, dan mengelola container secara konsisten. Docker memanfaatkan konsep seperti image, container, namespace, dan cgroup untuk memberikan isolasi serta pengelolaan resource pada aplikasi. Pemahaman mengenai konsep containerization dan arsitektur Docker menjadi dasar penting sebelum menerapkan container dalam proses pengembangan, deployment, dan penerapan DevSecOps.

1.2 Tujuan

1. Menjelaskan perbedaan virtual machine dan container dari sisi isolasi, ukuran, startup time, dan overhead.
2. Mengidentifikasi komponen Docker: client, daemon, registry, image, container, network, dan volume.
3. Menginstal Docker Engine pada Ubuntu serta Docker Desktop pada Windows berbasis WSL2.
4. Menjalankan container pertama, melakukan inspeksi image, melihat log, dan membangun image custom sederhana.

BAB II LANDASAN TEORI

2.1 Containerization

Containerization merupakan teknologi yang digunakan untuk menjalankan aplikasi beserta seluruh dependensinya dalam lingkungan yang terisolasi. Dengan container, aplikasi dapat dikemas dalam sebuah image sehingga dapat dijalankan secara konsisten pada berbagai lingkungan tanpa harus menginstal seluruh dependensi secara langsung pada sistem host.

2.2 Namespace

Namespace merupakan mekanisme pada sistem operasi Linux yang digunakan untuk memberikan lingkungan terisolasi kepada proses yang berjalan di dalam container. Dengan namespace, container dapat memiliki pandangan tersendiri terhadap berbagai resource sistem seperti proses, filesystem, network, hostname, dan user. Mekanisme ini membuat proses di dalam container seolah-olah berjalan pada lingkungan sistem yang terpisah meskipun sebenarnya masih menggunakan kernel yang sama dengan host.

2.3 Cgroup

Control Groups atau cgroup merupakan mekanisme Linux yang digunakan untuk mengatur dan membatasi penggunaan resource oleh sekumpulan proses. Pada container, cgroup dapat digunakan untuk membatasi resource seperti CPU, memori, I/O, dan jumlah proses yang dapat digunakan. Dengan adanya cgroup, penggunaan resource oleh sebuah container dapat dikendalikan sehingga tidak menghabiskan resource host secara berlebihan dan mengganggu container atau proses lainnya.

2.4 Docker

Docker merupakan platform yang digunakan untuk membuat, menjalankan, dan mengelola container. Docker menyediakan berbagai komponen seperti Docker Engine, Docker CLI, image, dan container yang memungkinkan proses deployment aplikasi dilakukan secara lebih mudah, konsisten, dan terisolasi.

BAB III Pengerjaan Praktikum

3.1 Container Nginx dan Ubuntu interaktif

- docker compose ps atau docker ps yang menunjukkan service berjalan.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0e6a95fba787	nginx:1.26	"/docker-entrypoint..."	About a minute ago	Up About a minute	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	web-public

- Screenshot hasil curl/browser/API/dashboard sesuai target bab.

```
adam@Devsecops:~/docker-lab/bab-2$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

- Cuplikan log atau query yang membuktikan sistem bekerja.

```
adam@Devsecops:~/docker-lab/bab-2$ docker logs --tail 20 web-public
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/08 00:07:35 [notice] 1#1: using the "epoll" event method
2026/09/08 00:07:35 [notice] 1#1: nginx/1.26.3
2026/09/08 00:07:35 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14)
2026/09/08 00:07:35 [notice] 1#1: OS: Linux 6.8.0-138-generic
2026/09/08 00:07:35 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/09/08 00:07:35 [notice] 1#1: start worker processes
2026/09/08 00:07:35 [notice] 1#1: start worker process 29
2026/09/08 00:07:35 [notice] 1#1: start worker process 30
2026/09/08 00:07:35 [notice] 1#1: start worker process 31
2026/09/08 00:07:35 [notice] 1#1: start worker process 32
```

3.2 Dockerfile custom web statis

- docker compose ps atau docker ps yang menunjukkan service berjalan

```
adam@Devsecops: ~/docker-lab/custom-web$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
5cd1ccda2b94   pens-web:1.0   "/docker-entrypoint..." 4 minutes ago  Up 4 minutes  0.0.0.0:9090->80/tcp, [::]:9090->80/tcp  pens-app
<h1>Docker Lab PENS</h1>
<p>Container berhasil berjalan.</p>
```

- Screenshot hasil curl/browser/API/dashboard sesuai target bab

```
adam@Devsecops: ~/docker-lab/custom-web$ docker version
Client: Docker Engine - Community
Version: 29.8.0
API version: 1.56
Go version: go1.26.8
Git commit: 88096ef
Built: Thu Sep 3 21:50:45 2026
OS/Arch: linux/amd64
Context: default

Server: Docker Engine - Community
Engine:
Version: 29.8.0
API version: 1.56 (minimum version 1.40)
Go version: go1.26.8
Git commit: 3ce5872
Built: Thu Sep 3 21:50:45 2026
OS/Arch: linux/amd64
Experimental: false
containerd:
Version: v2.3.4
GitCommit: db8809540e1a7a9da5d518876894933ff55692ab
runc:
Version: 1.5.1
GitCommit: v1.5.1-0-g8f2685a4
docker-init:
Version: 0.19.0
GitCommit: de40ad0

adam@Devsecops: ~/docker-lab/custom-web$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
5cd1ccda2b94   pens-web:1.0   "/docker-entrypoint..." 9 minutes ago  Up 9 minutes  0.0.0.0:9090->80/tcp, [::]:9090->80/tcp  pens-app
<h1>Docker Lab PENS</h1>
<p>Container berhasil berjalan.</p>
```

- Cuplikan log atau query yang membuktikan sistem bekerja.

```
adam@Devsecops: ~/docker-lab/custom-web$ docker logs --tail 20 pens-app
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/08 00:49:02 [notice] 1#1: using the "epoll" event method
2026/09/08 00:49:02 [notice] 1#1: nginx/1.26.3
2026/09/08 00:49:02 [notice] 1#1: built by gcc 13.2.1 20240309 (Alpine 13.2.1_git20240309)
2026/09/08 00:49:02 [notice] 1#1: OS: Linux 6.8.0-138-generic
2026/09/08 00:49:02 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/09/08 00:49:02 [notice] 1#1: start worker processes
2026/09/08 00:49:02 [notice] 1#1: start worker process 30
2026/09/08 00:49:02 [notice] 1#1: start worker process 31
2026/09/08 00:49:02 [notice] 1#1: start worker process 32
2026/09/08 00:49:02 [notice] 1#1: start worker process 33
172.17.0.1 - - [08/Sep/2026:00:53:53 +0000] "GET / HTTP/1.1" 200 61 "-" "curl/7.81.0" "-"
172.17.0.1 - - [08/Sep/2026:00:59:10 +0000] "GET / HTTP/1.1" 200 61 "-" "curl/7.81.0" "-"
```

3.3 Analisis wajib:

- Jelaskan satu masalah yang muncul dan cara Anda mendiagnosisnya.

Jawab: Salah satu masalah yang dapat muncul pada praktikum Docker adalah sayatidak memiliki permission untuk mengakses Docker daemon ketika menjalankan perintah **docker ps**. Diagnosis dilakukan dengan menjalankan `sudo docker ps`. Jika perintah tersebut berhasil, sedangkan `docker ps` tanpa `sudo` gagal, maka masalah berada pada permission pengguna terhadap Docker daemon. Selanjutnya dilakukan pengecekan menggunakan perintah `groups` dan pengguna ditambahkan ke group `docker` menggunakan `sudo usermod -aG docker $USER`. Setelah menjalankan `newgrp docker`, perintah Docker dapat digunakan tanpa `sudo`.

- Jelaskan risiko keamanan atau operasional yang relevan pada bab ini.

Jawab:

Risiko 1 — Docker group memiliki privilege tinggi

Risiko keamanan yang relevan adalah penggunaan Docker group yang memberikan akses tinggi terhadap Docker daemon. Jika akun pengguna tersebut disalahgunakan, penyerang berpotensi menjalankan container dengan privilege tinggi dan mengakses resource pada host..

```
sudo usermod -aG docker $USER
```

Risiko 2 — Menjalankan container sebagai root

Risiko keamanan lainnya adalah menjalankan aplikasi di dalam container menggunakan user `root`. Jika terdapat vulnerability pada aplikasi atau konfigurasi container, privilege yang tinggi dapat memperbesar dampak serangan terhadap host.

Risiko 3 — Image memiliki vulnerability

```
docker pull nginx:1.26  
FROM nginx:1.26-alpine
```

Penggunaan base image tanpa pemeriksaan vulnerability dapat menyebabkan container membawa package atau library yang memiliki kerentanan keamanan.

Risiko 4 — Data container tidak otomatis persistent

Untuk aplikasi production, data penting seharusnya menggunakan volume atau storage eksternal dan memiliki mekanisme backup.

- Berikan rekomendasi perbaikan bila lab ini akan dibawa ke production-like environment.

Jawab:

Jika lab ini diterapkan pada production-like environment, penggunaan Docker sebaiknya dilengkapi dengan beberapa pengamanan. Base image harus berasal dari sumber terpercaya, menggunakan versi yang terkontrol, dan dilakukan vulnerability scanning secara berkala. Aplikasi sebaiknya dijalankan menggunakan user non-root dan container diberikan batasan CPU serta memory. Secret seperti password dan API key tidak boleh ditanam langsung pada Dockerfile atau image. Data yang membutuhkan persistence harus menggunakan volume atau storage eksternal serta memiliki

mekanisme backup dan restore. Selain itu, pemeriksaan keamanan container sebaiknya diintegrasikan ke dalam CI/CD pipeline sehingga image hanya dapat dideploy setelah melewati pemeriksaan keamanan.

BAB IV KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Docker merupakan teknologi containerization yang dapat digunakan untuk menjalankan aplikasi secara terisolasi dan konsisten. Pada praktikum ini telah dilakukan instalasi dan verifikasi Docker Engine, menjalankan container hello-world dan Nginx, melakukan pemetaan port, serta membuat custom Docker image menggunakan Dockerfile. Container kemudian diverifikasi menggunakan `docker ps`, `docker logs`, dan `curl` untuk memastikan aplikasi dapat berjalan dengan baik. Selain memahami penggunaan image, container, Dockerfile, dan network, praktikum ini juga memberikan pemahaman mengenai pentingnya aspek keamanan seperti penggunaan image terpercaya, pembatasan privilege dan resource, serta pengelolaan data dan secret. Pengetahuan yang diperoleh menjadi dasar untuk menerapkan containerization dalam proses CI/CD dan lingkungan DevSecOps yang lebih aman dan terkontrol..